

Resource Quotas & Limit Ranges

Governing Namespace Resource Usage

Intermediate Kubernetes — Resource Governance (companion to Module 2)

Objectives

- Cap a namespace's aggregate compute and object counts with a **ResourceQuota**
- Understand why a quota **forces** Pods to declare requests/limits
- Supply per-container **defaults** and **min/max** with a **LimitRange**
- See how an aggregate quota **caps scaling**
- Distinguish `requests` (scheduling) from `limits` (enforcement)



Requests vs. Limits (Refresher)

requests

- What the container is **guaranteed**
- Used by the **scheduler** to place the Pod
- The number quotas count against

limits

- The **ceiling** the container may use
- Over memory limit → **OOMKilled**;
over CPU → throttled
- Enforced by the kubelet at runtime

Governance builds on these two numbers — a namespace with no requests can't be budgeted.



ResourceQuota

A namespace-wide budget

What a Quota Caps

Compute

- `requests.cpu / requests.memory`
- `limits.cpu / limits.memory`
- Extended resources (e.g. GPUs)

Object counts

- `pods, services, secrets`
- `persistentvolumeclaims, storage totals`
- `count/deployments.apps, services.loadbalancers`

Enforced at **admission**: `kubectl describe quota` shows Used vs Hard. Exceed it and the create is rejected with `exceeded quota`.

The Quota "Gotcha"

Once a namespace has a quota on `requests.cpu/memory`, **every** container must declare those requests — or the Pod is **rejected**.

failed quota: must specify requests.cpu, limits.cpu, ...

Kubernetes can't count what you don't declare. Forcing everyone to hand-write requests is painful — which is exactly what a **LimitRange** fixes.



LimitRange

Per-container defaults and bounds

What a LimitRange Does

Defaulting

- `defaultRequest` → a container's requests
- `default` → a container's limits
- Applied only when the container omits them

Bounding

- `min` / `max` per container
- Requests below `min` or above `max` → **rejected**
- Also covers PVC storage size

Defaulting makes the quota usable (no more hand-writing requests); `min/max` stops a single Pod hogging the whole budget.

How They Work Together

	ResourceQuota	LimitRange
Scope	Whole namespace (aggregate)	Each container / PVC
Job	Cap the <i>total</i>	Default & bound the <i>individual</i>
On violation	exceeded quota	min/max violation

Typical setup: one ResourceQuota + one LimitRange per tenant namespace, applied automatically (here, by a Kyverno generate policy).



Lab: Resource Quotas & Limit Ranges

Hands-on: govern a namespace's resources

- Create a **ResourceQuota** and read Used vs Hard
- Watch a no-resources Pod get **rejected** by the quota
- Add a **LimitRange** so the same Pod is admitted with **defaults**
- Push a Deployment past the quota and see it **capped** below its replica count

Duration: ~25-35 minutes

Key Takeaways

- A **ResourceQuota** caps a namespace's *aggregate* compute and object counts (Used vs Hard).
- A compute quota **requires** every container to declare requests/limits — or the Pod is rejected.
- A **LimitRange** supplies per-container **defaults** and enforces **min/max**.
- Aggregate quotas **cap scaling**: a Deployment stops at the replicas that fit, with exceeded quota.
- Quota + LimitRange = the foundation of **multi-tenant** resource governance.